direct_reconstruction

June 28, 2024

1 Direct Reconstruction of 2D golden angle radial data

Here we use the DirectReconstruction class to reconstruct images from ISMRMRD 2D radial data

1.1 Image reconstruction

We use the DirectReconstruction class to reconstruct images from 2D radial data. DirectReconstruction estimates CSMs, DCFs and performs an adjoint Fourier transform. This is a high-level interface to the reconstruction pipeline.

1.1.1 Import

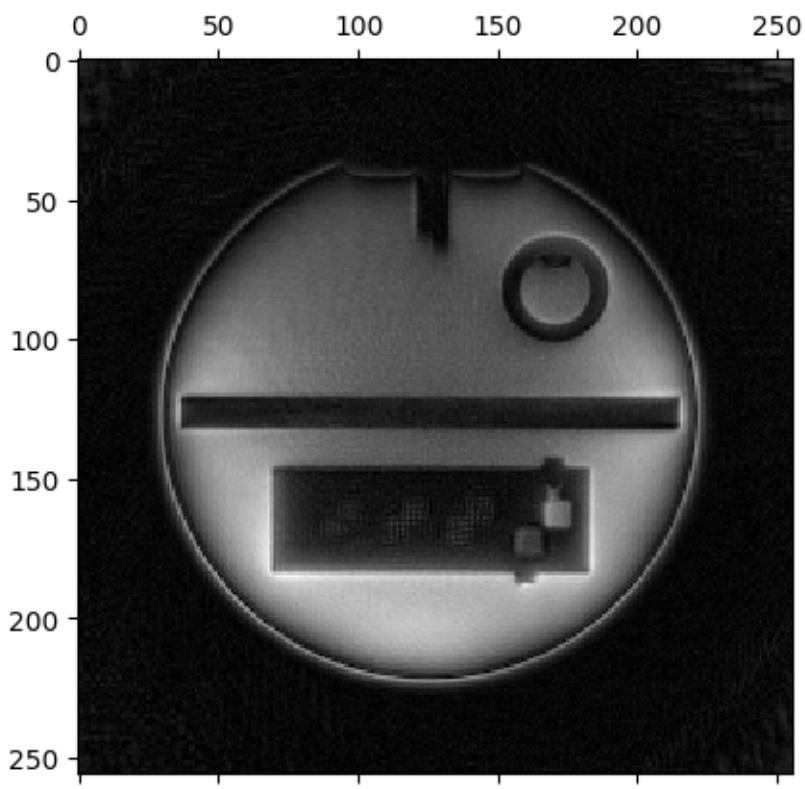
[ ]:
```
# define zenodo URL of the example ismrmd data
zenodo_url = 'https://zenodo.org/records/10854057/files/'
fname = 'meas_MID00177_FID02509_20240319_radial_2D_256mm_402spokes_uniform.h5'
```

[ ]:
```
# Import
import shutil
import torch
import tempfile
from pathlib import Path
from mrpro.data import KData  # Import the KData class
from mrpro.data.traj_calculators import KTrajectoryPulseq

import requests

# Local path
h5_path = '/home/bouill01/data/
 ↪20240319_spiral_2D_256mm_220k0_128interleaves_golden_angle_vds/
 ↪pulseq_spiral_2D_220k0_128interleaves_golden_angle_vds_with_traj.h5'
seq_path = '/home/bouill01/data/
 ↪20240319_spiral_2D_256mm_220k0_128interleaves_golden_angle_vds/
 ↪20240319_spiral_2D_256mm_220k0_128interleaves_golden_angle_vds.seq'
```

1.1.2 Direct reconstruction using KTrajectoryIsmrmrd

[ ]:
```
import mrpro
from mrpro.data.traj_calculators import KTrajectoryIsmrmrd
import matplotlib.pyplot as plt

# Use the trajectory that is stored in the ISMRMRD file
# trajectory = mrpro.data.traj_calculators.KTrajectoryIsmrmrd(seq_file)
# Load in the Data from the ISMRMRD file
kdata = KData.from_file(h5_path, KTrajectoryIsmrmrd())
# Perform the reconstruction
reconstruction = mrpro.algorithms.reconstruction.DirectReconstruction.
 ↪from_kdata(kdata)
# Use this to run on gpu: kdata = kdata.cuda()
img = reconstruction(kdata)

# Display the reconstructed image
# If there are multiple slices, ..., only the first one is selected
first_img = img.rss().cpu()[0, 0, :, :]  #  images, z, y, x
plt.matshow(first_img, cmap='gray')
```

[ ]:
```
<matplotlib.image.AxesImage at 0x7f1220522950>
```



1.1.3 Direct reconstruction using KTrajectoryPulseq

Definition of the shifting fonction

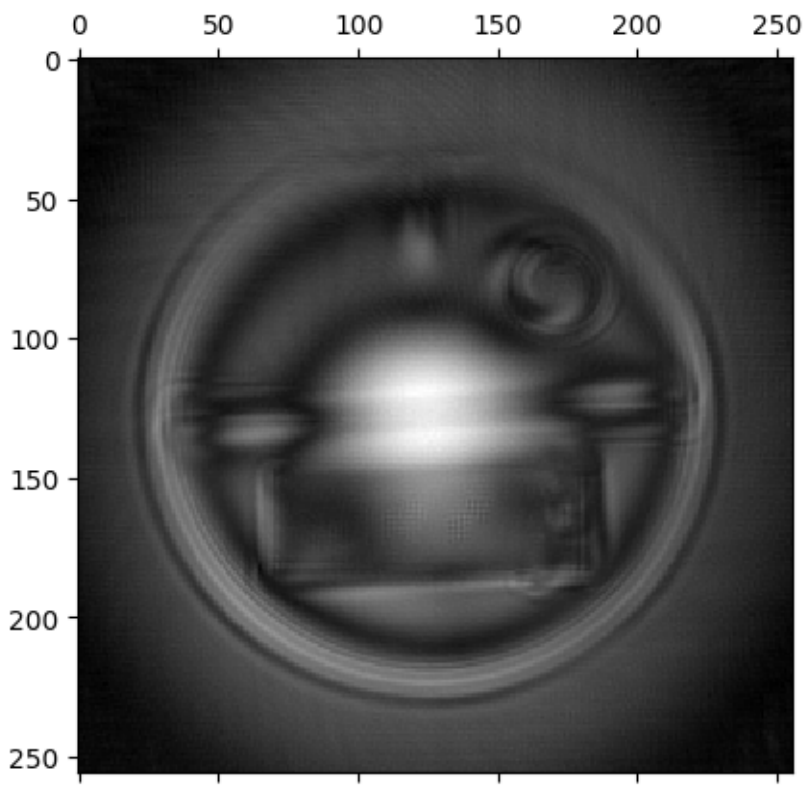
[ ]:
```
import mrpro
from mrpro.data import KData, KTrajectory
from mrpro.data.traj_calculators import KTrajectoryPulseq
import matplotlib.pyplot as plt
import numpy as np

def shift_k_space_trajectory(kdatapuls):
    # Extract k-space trajectory from kdatapuls
    ky_pulseq = kdatapuls.traj.ky
    kx_pulseq = kdatapuls.traj.kx
    kz_pulseq = kdatapuls.traj.kz

    # Number of indices
    num_indices = ky_pulseq.shape[2]

    # Initialize lists to store shifted trajectories
    shifted_ky = ky_pulseq.clone()
    shifted_kx = kx_pulseq.clone()

    # Loop to apply the shift to each index
    for i in range(num_indices-1):
        # Calculate the shift for the current index
        shifted_ky[:,:,i,:] -= ky_pulseq[:,:,i,0]
        shifted_kx[:,:,i,:] -= kx_pulseq[:,:,i,0]

    # Create shifted KTrajectory object
    shifted_traj = KTrajectory(kx=shifted_kx, ky=shifted_ky, kz=kz_pulseq)
    # Create shifted KData object
    shifted_kdatapuls = KData(data=kdatapuls.data, traj=shifted_traj,␣
 ↪header=kdatapuls.header)

    return shifted_kdatapuls
```

Before shifting

[ ]:
```
import mrpro
from mrpro.data.traj_calculators import KTrajectoryIsmrmrd

# Use the trajectory that is stored in the ISMRMRD file
# trajectory = mrpro.data.traj_calculators.KTrajectoryIsmrmrd(seq_file)
# Load in the Data from the ISMRMRD file
kdatapuls = KData.from_file(h5_path, KTrajectoryPulseq(seq_path=seq_path))
# Perform the reconstruction
```

```
reconstructionpuls = mrpro.algorithms.reconstruction.DirectReconstruction.
 ↪from_kdata(kdatapuls)
# Use this to run on gpu: kdata = kdata.cuda()
img = reconstructionpuls(kdatapuls)

import matplotlib.pyplot as plt

# Display the reconstructed image
# If there are multiple slices, ..., only the first one is selected
first_img = img.rss().cpu()[0, 0, :, :]  #  images, z, y, x
plt.matshow(first_img, cmap='gray')
```

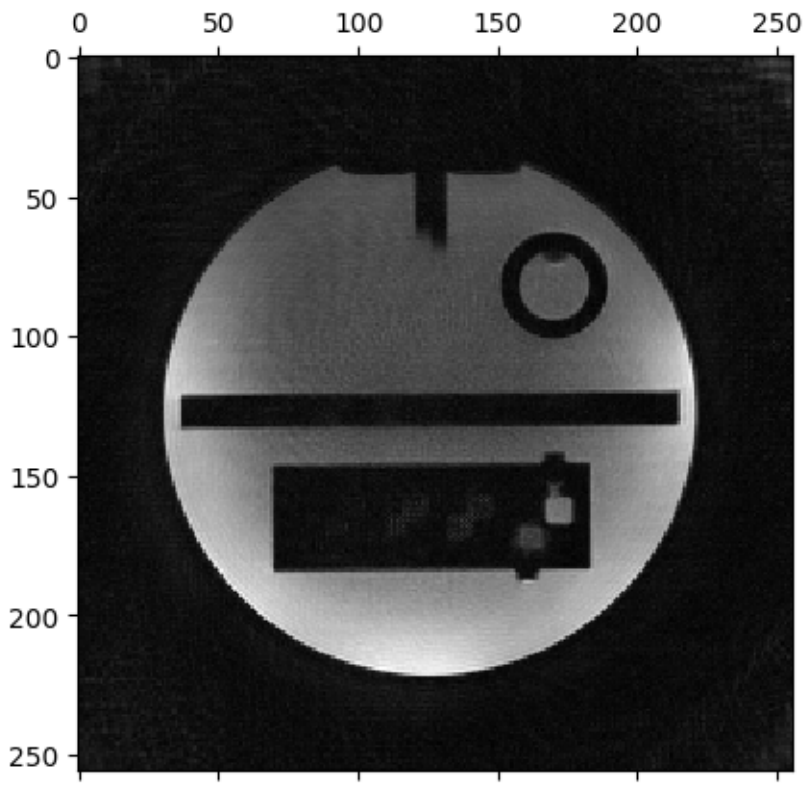
[ ]: <matplotlib.image.AxesImage at 0x7f122024d290>



After shifting

[ ]:
```
# Load data from Pulseq file using KTrajectoryPulseq
kdatapuls = KData.from_file(h5_path, KTrajectoryPulseq(seq_path=seq_path))

# Apply the shift to the k-space data
shifted_kdatapuls = shift_k_space_trajectory(kdatapuls)
```

```
# Perform the reconstruction
reconstructionpuls = mrpro.algorithms.reconstruction.DirectReconstruction.
 ↪from_kdata(shifted_kdatapuls, noise=None, coil_combine=None)
img = reconstructionpuls(kdatapuls)

# Display the reconstructed image
first_img = img.rss().cpu()[0, 0, :, :]  # images, z, y, x
plt.matshow(first_img, cmap='gray')
plt.show()
```



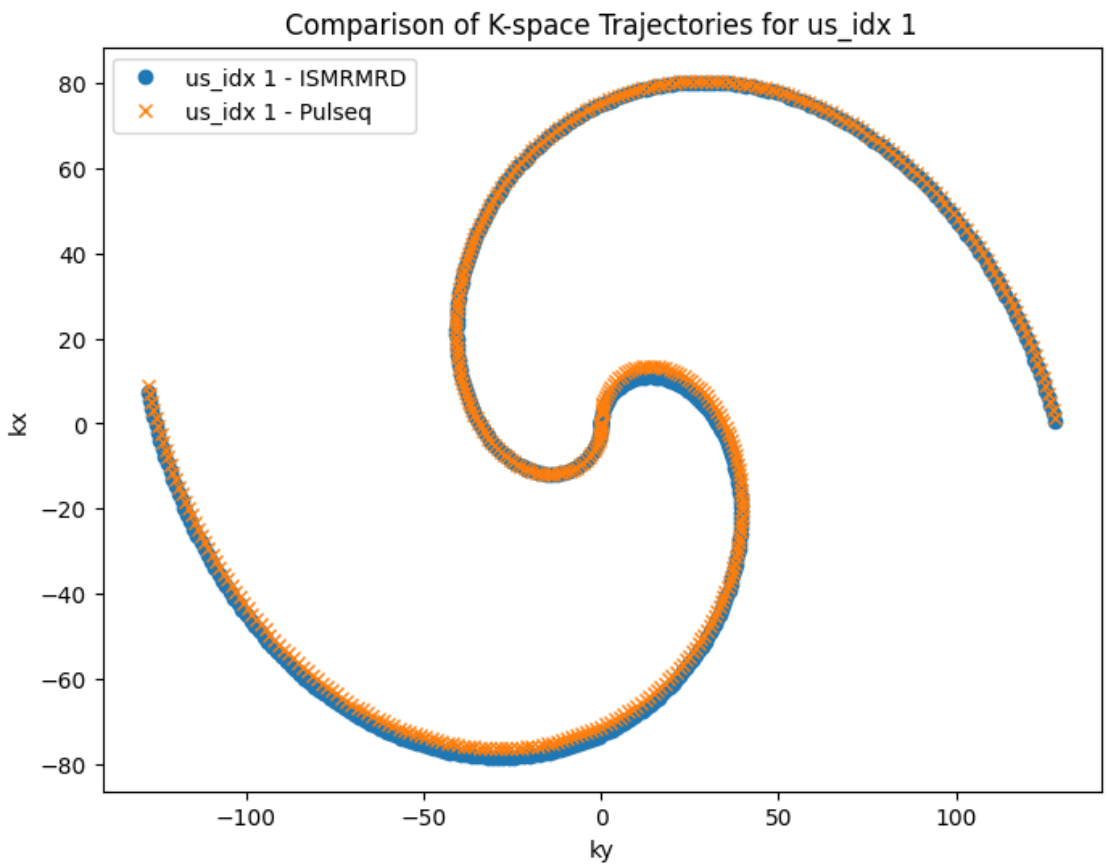
1.1.4 Trajectories Pulseq/ISMRMRD

Before shifting

[ ]:
```
import torch
from mrpro.data import KData
from mrpro.data.traj_calculators import KTrajectoryIsmrmrd, KTrajectoryPulseq
import matplotlib.pyplot as plt

# Load data from ISMRMRD file using KTrajectoryIsmrmrd
kdata_ismrmrd = KData.from_file(h5_path, KTrajectoryIsmrmrd())
```

```python
# Load data from Pulseq file using KTrajectoryPulseq
kdata_pulseq = KData.from_file(h5_path, KTrajectoryPulseq(seq_path=seq_path))

# Define values to compare for us_idx
us_idx_values = [torch.arange(0, 128, 127)[None, :]]

# For each us_idx value, plot the first and last points of the k-space␣
↪trajectory
for i, us_idx in enumerate(us_idx_values):
    print(f"Calculating for us_idx {i+1}:", us_idx)

    # Split k-space data into other dimensions based on undersampling indices
    kdata_us_ismrmrd = kdata_ismrmrd.split_k1_into_other(us_idx,␣
↪other_label='repetition')
    kdata_us_pulseq = kdata_pulseq.split_k1_into_other(us_idx,␣
↪other_label='repetition')

    # Plot k-space trajectory for ISMRMRD
    plt.figure(figsize=(8, 6))
    plt.plot(kdata_us_ismrmrd.traj.ky.flatten(), kdata_us_ismrmrd.traj.kx.
↪flatten(), 'o', label=f'us_idx {i+1} - ISMRMRD')

    # Plot k-space trajectory for Pulseq
    plt.plot(kdata_us_pulseq.traj.ky.flatten(), kdata_us_pulseq.traj.kx.
↪flatten(), 'x', label=f'us_idx {i+1} - Pulseq')

    # Add labels and legend
    plt.xlabel('ky')
    plt.ylabel('kx')
    plt.title(f'Comparison of K-space Trajectories for us_idx {i+1}')
    plt.legend()
    plt.show()
```

```
Calculating for us_idx 1: tensor([[  0, 127]])
```

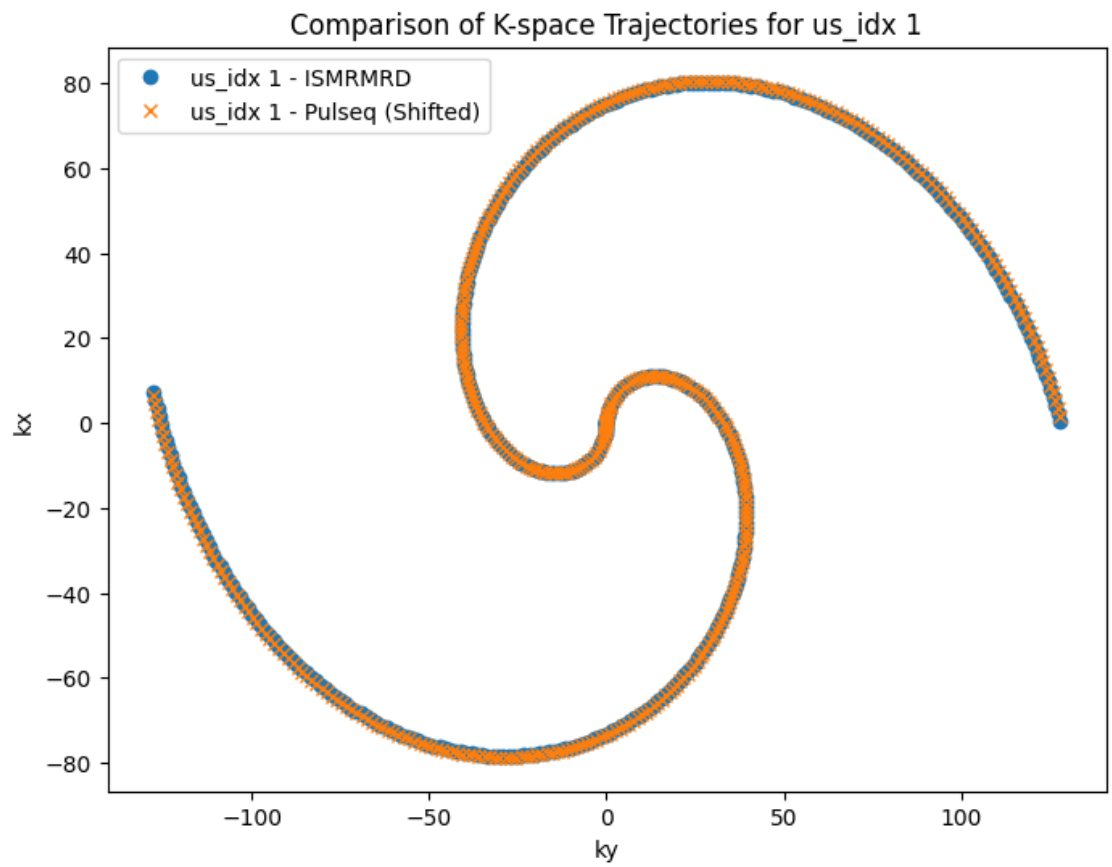


After shifting (flatten)

[ ]:
```
import torch
from mrpro.data import KData
from mrpro.data.traj_calculators import KTrajectoryIsmrmrd, KTrajectoryPulseq
import matplotlib.pyplot as plt

# Load data from ISMRMRD file using KTrajectoryIsmrmrd
kdata_ismrmrd = KData.from_file(h5_path, KTrajectoryIsmrmrd())

# Load data from Pulseq file using KTrajectoryPulseq
kdata_pulseq = KData.from_file(h5_path, KTrajectoryPulseq(seq_path=seq_path))

# Define values to compare for us_idx
us_idx_values = [torch.arange(0, 128, 127)[None, :]]

# For each us_idx value, plot the first and last points of the k-space␣
↪trajectory
for i, us_idx in enumerate(us_idx_values):
    print(f"Calculating for us_idx {i+1}:", us_idx)
```

```
    # Split k-space data into other dimensions based on undersampling indices
    kdata_us_ismrmrd = kdata_ismrmrd.split_k1_into_other(us_idx,␣
↪other_label='repetition')
    kdata_us_pulseq = kdata_pulseq.split_k1_into_other(us_idx,␣
↪other_label='repetition')

    # Shift the k-space trajectory for index 0
    shifted_ky_pulseq = kdata_us_pulseq.traj.ky.flatten() - kdata_us_pulseq.
↪traj.ky.flatten()[0]
    shifted_kx_pulseq = kdata_us_pulseq.traj.kx.flatten() - kdata_us_pulseq.
↪traj.kx.flatten()[0]

    # Shift the k-space trajectory for index 127
    shifted_ky_pulseq[220:] -= kdata_us_pulseq.traj.ky.flatten()[220]
    shifted_kx_pulseq[220:] -= kdata_us_pulseq.traj.kx.flatten()[220]

    # Plot k-space trajectory for ISMRMRD
    plt.figure(figsize=(8, 6))
    plt.plot(kdata_us_ismrmrd.traj.ky.flatten(), kdata_us_ismrmrd.traj.kx.
↪flatten(), 'o', label=f'us_idx {i+1} - ISMRMRD')

    # Plot shifted k-space trajectory for Pulseq
    plt.plot(shifted_ky_pulseq, shifted_kx_pulseq, 'x', label=f'us_idx {i+1} -␣
↪Pulseq (Shifted)')

    # Add labels and legend
    plt.xlabel('ky')
    plt.ylabel('kx')
    plt.title(f'Comparison of K-space Trajectories for us_idx {i+1}')
    plt.legend()
    plt.show()
```

```
Calculating for us_idx 1: tensor([[  0, 127]])
```

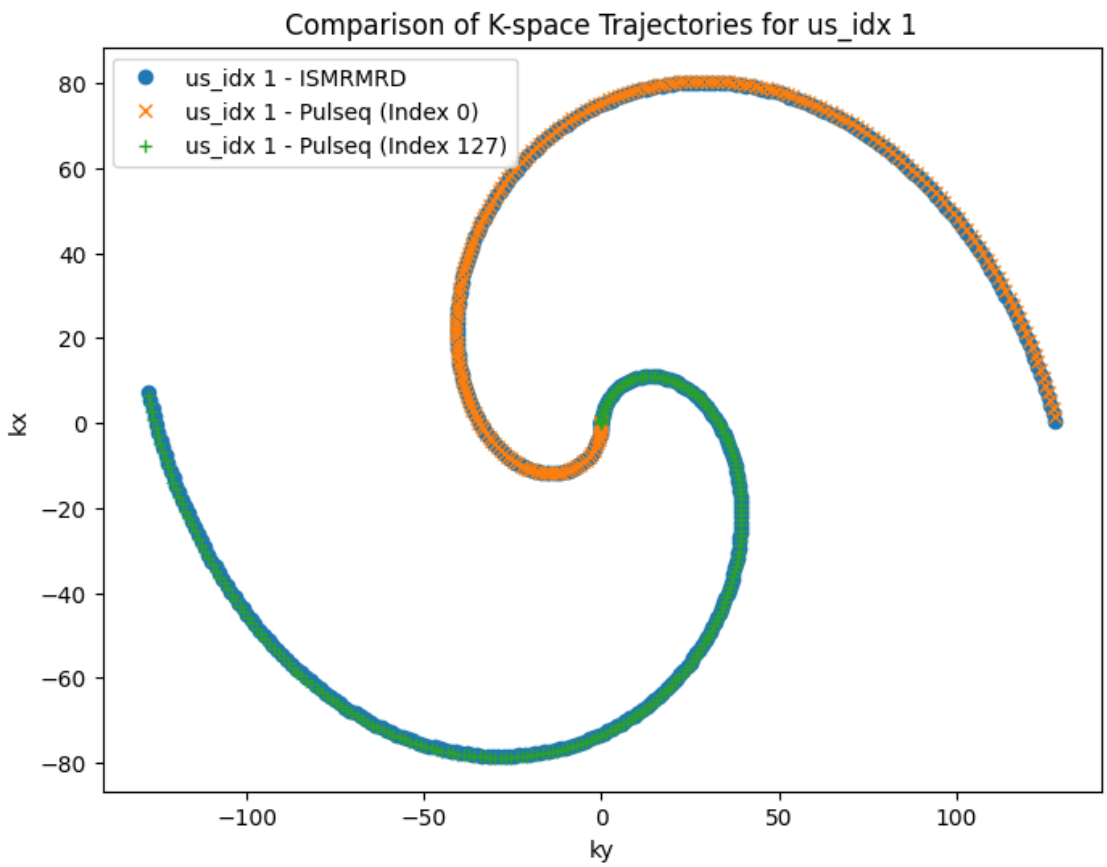


After shifting (traj separate)

[ ]:
```
import torch
from mrpro.data import KData
from mrpro.data.traj_calculators import KTrajectoryIsmrmrd, KTrajectoryPulseq
import matplotlib.pyplot as plt

# Load data from ISMRMRD file using KTrajectoryIsmrmrd
kdata_ismrmrd = KData.from_file(h5_path, KTrajectoryIsmrmrd())

# Load data from Pulseq file using KTrajectoryPulseq
kdata_pulseq = KData.from_file(h5_path, KTrajectoryPulseq(seq_path=seq_path))

# Define values to compare for us_idx
us_idx_values = [torch.arange(0, 128, 127)[None, :]]

# For each us_idx value, plot the first and last points of the k-space␣
↪trajectory
for i, us_idx in enumerate(us_idx_values):
    print(f"Calculating for us_idx {i+1}:", us_idx)
```

```
    # Split k-space data into other dimensions based on undersampling indices
    kdata_us_ismrmrd = kdata_ismrmrd.split_k1_into_other(us_idx,␣
↪other_label='repetition')
    kdata_us_pulseq = kdata_pulseq.split_k1_into_other(us_idx,␣
↪other_label='repetition')

    # Extract the trajectories for index 0 and 127
    traj_ky_0 = kdata_us_pulseq.traj.ky[:, :, 0, :]
    traj_ky_127 = kdata_us_pulseq.traj.ky[:, :, -1, :]
    traj_kx_0 = kdata_us_pulseq.traj.kx[:, :, 0, :]
    traj_kx_127 = kdata_us_pulseq.traj.kx[:, :, -1, :]

    # Shift the k-space trajectories for index 0
    shifted_ky_pulseq_0 = traj_ky_0 - traj_ky_0[:,:, 0, None]
    shifted_kx_pulseq_0 = traj_kx_0 - traj_kx_0[:,:, 0, None]

    # Shift the k-space trajectories for index 127
    shifted_ky_pulseq_127 = traj_ky_127 - traj_ky_127[:,:,0, None]
    shifted_kx_pulseq_127 = traj_kx_127 - traj_kx_127[:,:,0, None]

    # Plot k-space trajectory for ISMRMRD
    plt.figure(figsize=(8, 6))
    plt.plot(kdata_us_ismrmrd.traj.ky.flatten(), kdata_us_ismrmrd.traj.kx.
↪flatten(), 'o', label=f'us_idx {i+1} - ISMRMRD')

    # Plot shifted k-space trajectory for Pulseq (index 0)
    plt.plot(shifted_ky_pulseq_0.cpu().numpy().flatten(), shifted_kx_pulseq_0.
↪cpu().numpy().flatten(), 'x', label=f'us_idx {i+1} - Pulseq (Index 0)')

    # Plot shifted k-space trajectory for Pulseq (index 127)
    plt.plot(shifted_ky_pulseq_127.cpu().numpy().flatten(),␣
↪shifted_kx_pulseq_127.cpu().numpy().flatten(), '+', label=f'us_idx {i+1} -␣
↪Pulseq (Index 127)')

    # Add labels and legend
    plt.xlabel('ky')
    plt.ylabel('kx')
    plt.title(f'Comparison of K-space Trajectories for us_idx {i+1}')
    plt.legend()
    plt.show()
```

```
Calculating for us_idx 1: tensor([[  0, 127]])
```



1.1.5 Distance between trajectories pulseq/ISMRMRD

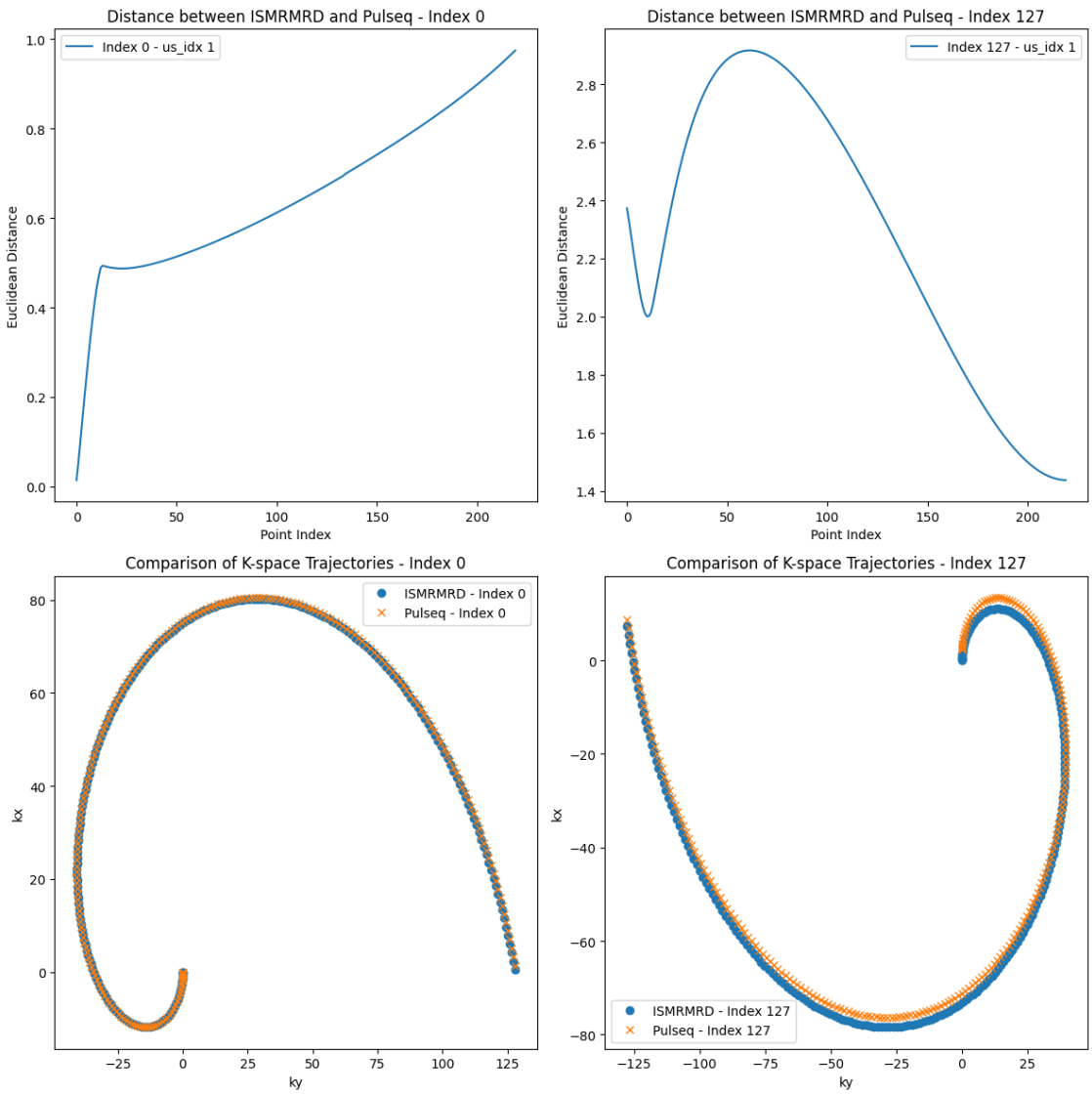
Before shifting

[ ]:
```
import torch
from mrpro.data import KData
from mrpro.data.traj_calculators import KTrajectoryIsmrmrd, KTrajectoryPulseq
import matplotlib.pyplot as plt

# Load data from ISMRMRD file using KTrajectoryIsmrmrd
kdata_ismrmrd = KData.from_file(h5_path, KTrajectoryIsmrmrd())

# Load data from Pulseq file using KTrajectoryPulseq
kdata_pulseq = KData.from_file(h5_path, KTrajectoryPulseq(seq_path=seq_path))

# Define values to compare for us_idx
us_idx_values = [torch.arange(0, 128, 127)[None, :]]

# Create a 2x2 grid of subplots
fig, axs = plt.subplots(2, 2, figsize=(12, 12))
```

```
# For each us_idx value, plot the first and last points of the k-space␣
↪trajectory
for i, us_idx in enumerate(us_idx_values):
    print(f"Calculating for us_idx {i+1}:", us_idx)

    # Split k-space data into other dimensions based on undersampling indices
    kdata_us_ismrmrd = kdata_ismrmrd.split_k1_into_other(us_idx,␣
↪other_label='repetition')
    kdata_us_pulseq = kdata_pulseq.split_k1_into_other(us_idx,␣
↪other_label='repetition')

    # Extract the trajectories
    traj_ky_ismrmrd = kdata_us_ismrmrd.traj.ky
    traj_kx_ismrmrd = kdata_us_ismrmrd.traj.kx
    traj_ky_pulseq = kdata_us_pulseq.traj.ky
    traj_kx_pulseq = kdata_us_pulseq.traj.kx

    # Calculate the difference between Pulseq and ISMRMRD trajectories
    diff_ky = traj_ky_pulseq - traj_ky_ismrmrd
    diff_kx = traj_kx_pulseq - traj_kx_ismrmrd

    # Calculate the Euclidean distance between the trajectories
    distance = torch.sqrt(diff_ky**2 + diff_kx**2).cpu().numpy()

    # Plot the distance for index 0
    axs[0, 0].plot(distance[:, :, 0].flatten(), label=f'Index 0 - us_idx {i+1}')
    axs[0, 0].set_xlabel('Point Index')
    axs[0, 0].set_ylabel('Euclidean Distance')
    axs[0, 0].set_title(f'Distance between ISMRMRD and Pulseq - Index 0')
    axs[0, 0].legend()

    # Plot the distance for index 127
    axs[0, 1].plot(distance[:, :, -1].flatten(), label=f'Index 127 - us_idx␣
↪{i+1}')
    axs[0, 1].set_xlabel('Point Index')
    axs[0, 1].set_ylabel('Euclidean Distance')
    axs[0, 1].set_title(f'Distance between ISMRMRD and Pulseq - Index 127')
    axs[0, 1].legend()

    # Plot k-space trajectories for ISMRMRD and Pulseq (index 0)
    axs[1, 0].plot(traj_ky_ismrmrd[:, :, 0].cpu().numpy().flatten(),␣
↪traj_kx_ismrmrd[:, :, 0].cpu().numpy().flatten(), 'o', label=f'ISMRMRD -␣
↪Index 0')
    axs[1, 0].plot(traj_ky_pulseq[:, :, 0].cpu().numpy().flatten(),␣
↪traj_kx_pulseq[:, :, 0].cpu().numpy().flatten(), 'x', label=f'Pulseq - Index␣
↪0')
```

```
    axs[1, 0].set_xlabel('ky')
    axs[1, 0].set_ylabel('kx')
    axs[1, 0].set_title(f'Comparison of K-space Trajectories - Index 0')
    axs[1, 0].legend()

    # Plot k-space trajectories for ISMRMRD and Pulseq (index 127)
    axs[1, 1].plot(traj_ky_ismrmrd[:, :, -1].cpu().numpy().flatten(),␣
↪traj_kx_ismrmrd[:, :, -1].cpu().numpy().flatten(), 'o', label=f'ISMRMRD -␣
↪Index 127')
    axs[1, 1].plot(traj_ky_pulseq[:, :, -1].cpu().numpy().flatten(),␣
↪traj_kx_pulseq[:, :, -1].cpu().numpy().flatten(), 'x', label=f'Pulseq -␣
↪Index 127')
    axs[1, 1].set_xlabel('ky')
    axs[1, 1].set_ylabel('kx')
    axs[1, 1].set_title(f'Comparison of K-space Trajectories - Index 127')
    axs[1, 1].legend()

# Adjust layout and display the plots
plt.tight_layout()
plt.show()
```

```
Calculating for us_idx 1: tensor([[  0, 127]])
```



After shifting

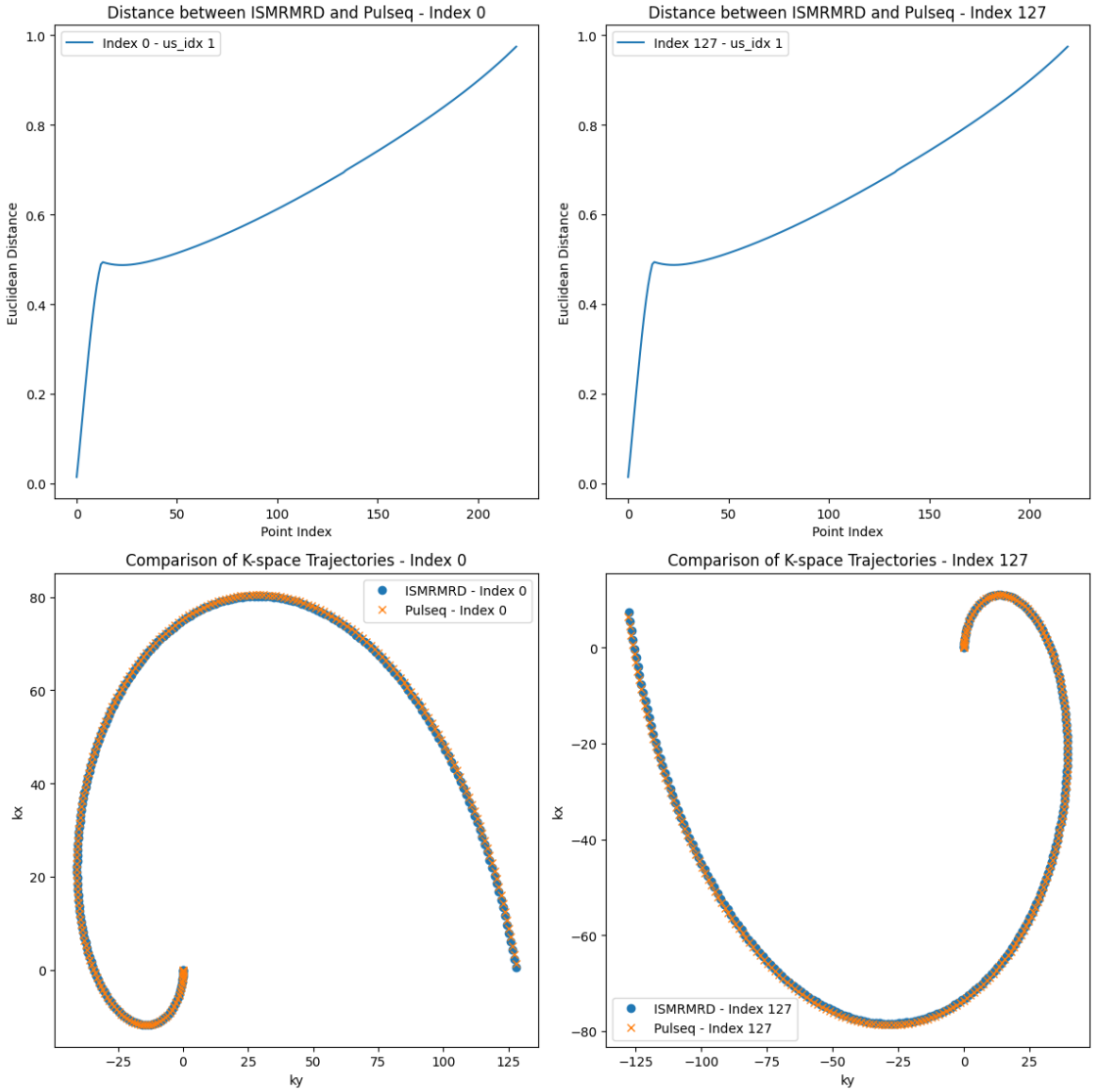
[ ]:
```
import torch
from mrpro.data import KData
from mrpro.data.traj_calculators import KTrajectoryIsmrmrd, KTrajectoryPulseq
import matplotlib.pyplot as plt

# Load data from ISMRMRD file using KTrajectoryIsmrmrd
kdata_ismrmrd = KData.from_file(h5_path, KTrajectoryIsmrmrd())

# Load data from Pulseq file using KTrajectoryPulseq
kdata_pulseq = KData.from_file(h5_path, KTrajectoryPulseq(seq_path=seq_path))
```

```python
# Define values to compare for us_idx
us_idx_values = [torch.arange(0, 128, 127)[None, :]]

# Create a 2x2 grid of subplots
fig, axs = plt.subplots(2, 2, figsize=(12, 12))

# For each us_idx value, plot the first and last points of the k-space␣
↪trajectory
for i, us_idx in enumerate(us_idx_values):
    print(f"Calculating for us_idx {i+1}:", us_idx)

    # Split k-space data into other dimensions based on undersampling indices
    kdata_us_ismrmrd = kdata_ismrmrd.split_k1_into_other(us_idx,␣
↪other_label='repetition')
    kdata_us_pulseq = kdata_pulseq.split_k1_into_other(us_idx,␣
↪other_label='repetition')

    # Extract the trajectories for index 0 and 127
    traj_ky_0 = kdata_us_pulseq.traj.ky[:, :, 0, :]
    traj_ky_127 = kdata_us_pulseq.traj.ky[:, :, -1, :]
    traj_kx_0 = kdata_us_pulseq.traj.kx[:, :, 0, :]
    traj_kx_127 = kdata_us_pulseq.traj.kx[:, :, -1, :]
    print(f'traj_ky_0_0 = {traj_ky_0[0,0,0]}')
    print(f'traj_kx_0_0 = {traj_kx_0[0,0,0]}')
    print(f'traj_ky_127_0 = {traj_ky_127[0,0,0]}')
    print(f'traj_kx_127_0 = {traj_kx_127[0,0,0]}')
    # Shift the k-space trajectories for index 0
    shifted_ky_pulseq_0 = traj_ky_0 - traj_ky_0[:,:, 0, None]
    shifted_kx_pulseq_0 = traj_kx_0 - traj_kx_0[:,:, 0, None]

    # Shift the k-space trajectories for index 127
    shifted_ky_pulseq_127 = traj_ky_127 - traj_ky_127[:,:,0, None]
    shifted_kx_pulseq_127 = traj_kx_127 - traj_kx_127[:,:,0, None]

    print(f'shifted_ky_0_0 = {shifted_ky_pulseq_0[0,0,0]}')
    print(f'shifted_kx_0_0 = {shifted_kx_pulseq_0[0,0,0]}')
    print(f'shifted_ky_127_0 = {shifted_ky_pulseq_127[0,0,0]}')
    print(f'shifted_kx_127_0 = {shifted_kx_pulseq_127[0,0,0]}')

    # Combine the shifted trajectories into one tensor
    traj_ky_shiftpulseq = torch.cat((shifted_ky_pulseq_0.unsqueeze(2),␣
↪shifted_ky_pulseq_127.unsqueeze(2)), dim=2)
    traj_kx_shiftpulseq = torch.cat((shifted_kx_pulseq_0.unsqueeze(2),␣
↪shifted_kx_pulseq_127.unsqueeze(2)), dim=2)

    # Extract the trajectories for ISMRMRD
    traj_ky_ismrmrd = kdata_us_ismrmrd.traj.ky
```

```
    traj_kx_ismrmrd = kdata_us_ismrmrd.traj.kx

    print(f'irmrmrd_ky_0_0 = {traj_ky_ismrmrd[0,0,0,0]}')
    print(f'irmrmrd_kx_0_0 = {traj_kx_ismrmrd[0,0,0,0]}')
    print(f'irmrmrd_ky_127_0 = {traj_ky_ismrmrd[0,0,1,0]}')
    print(f'irmrmrd_kx_127_0 = {traj_kx_ismrmrd[0,0,1,0]}')

    # Calculate the difference between Pulseq and ISMRMRD trajectories
    diff_ky = traj_ky_shiftpulseq - traj_ky_ismrmrd
    diff_kx = traj_kx_shiftpulseq - traj_kx_ismrmrd

    # Calculate the Euclidean distance between the trajectories
    distance = torch.sqrt(diff_ky**2 + diff_kx**2).cpu().numpy()

    # Plot the distance for index 0
    axs[0, 0].plot(distance[:, :, 0].flatten(), label=f'Index 0 - us_idx {i+1}')
    axs[0, 0].set_xlabel('Point Index')
    axs[0, 0].set_ylabel('Euclidean Distance')
    axs[0, 0].set_title(f'Distance between ISMRMRD and Pulseq - Index 0')
    axs[0, 0].legend()

    # Plot the distance for index 127
    axs[0, 1].plot(distance[:, :, 1].flatten(), label=f'Index 127 - us_idx␣
↪{i+1}')
    axs[0, 1].set_xlabel('Point Index')
    axs[0, 1].set_ylabel('Euclidean Distance')
    axs[0, 1].set_title(f'Distance between ISMRMRD and Pulseq - Index 127')
    axs[0, 1].legend()

    # Plot k-space trajectories for ISMRMRD and Pulseq (index 0)
    axs[1, 0].plot(traj_ky_ismrmrd[:, :, 0].cpu().numpy().flatten(),␣
↪traj_kx_ismrmrd[:, :, 0].cpu().numpy().flatten(), 'o', label=f'ISMRMRD -␣
↪Index 0')
    axs[1, 0].plot(traj_ky_shiftpulseq[:, :, 0].cpu().numpy().flatten(),␣
↪traj_kx_shiftpulseq[:, :, 0].cpu().numpy().flatten(), 'x', label=f'Pulseq -␣
↪Index 0')
    axs[1, 0].set_xlabel('ky')
    axs[1, 0].set_ylabel('kx')
    axs[1, 0].set_title(f'Comparison of K-space Trajectories - Index 0')
    axs[1, 0].legend()

    # Plot k-space trajectories for ISMRMRD and Pulseq (index 127)
    axs[1, 1].plot(traj_ky_ismrmrd[:, :, -1].cpu().numpy().flatten(),␣
↪traj_kx_ismrmrd[:, :, -1].cpu().numpy().flatten(), 'o', label=f'ISMRMRD -␣
↪Index 127')
```

```
    axs[1, 1].plot(traj_ky_shiftpulseq[:, :, 1].cpu().numpy().flatten(),␣
↪traj_kx_shiftpulseq[:, :, 1].cpu().numpy().flatten(), 'x', label=f'Pulseq -␣
↪Index 127')
    axs[1, 1].set_xlabel('ky')
    axs[1, 1].set_ylabel('kx')
    axs[1, 1].set_title(f'Comparison of K-space Trajectories - Index 127')
    axs[1, 1].legend()

# Adjust layout and display the plots
plt.tight_layout()
plt.show()
```

```
Calculating for us_idx 1: tensor([[  0, 127]])
traj_ky_0_0 = -5.041608948895548e-13
traj_kx_0_0 = -7.180124272920807e-10
traj_ky_127_0 = 0.14733916521072388
traj_kx_127_0 = 2.3829565048217773
shifted_ky_0_0 = 0.0
shifted_kx_0_0 = 0.0
shifted_ky_127_0 = 0.0
shifted_kx_127_0 = 0.0
irmrmrd_ky_0_0 = -1.0045425369753502e-05
irmrmrd_kx_0_0 = -0.014305529184639454
irmrmrd_ky_127_0 = 0.0008798780618235469
irmrmrd_kx_127_0 = 0.014278448186814785
```

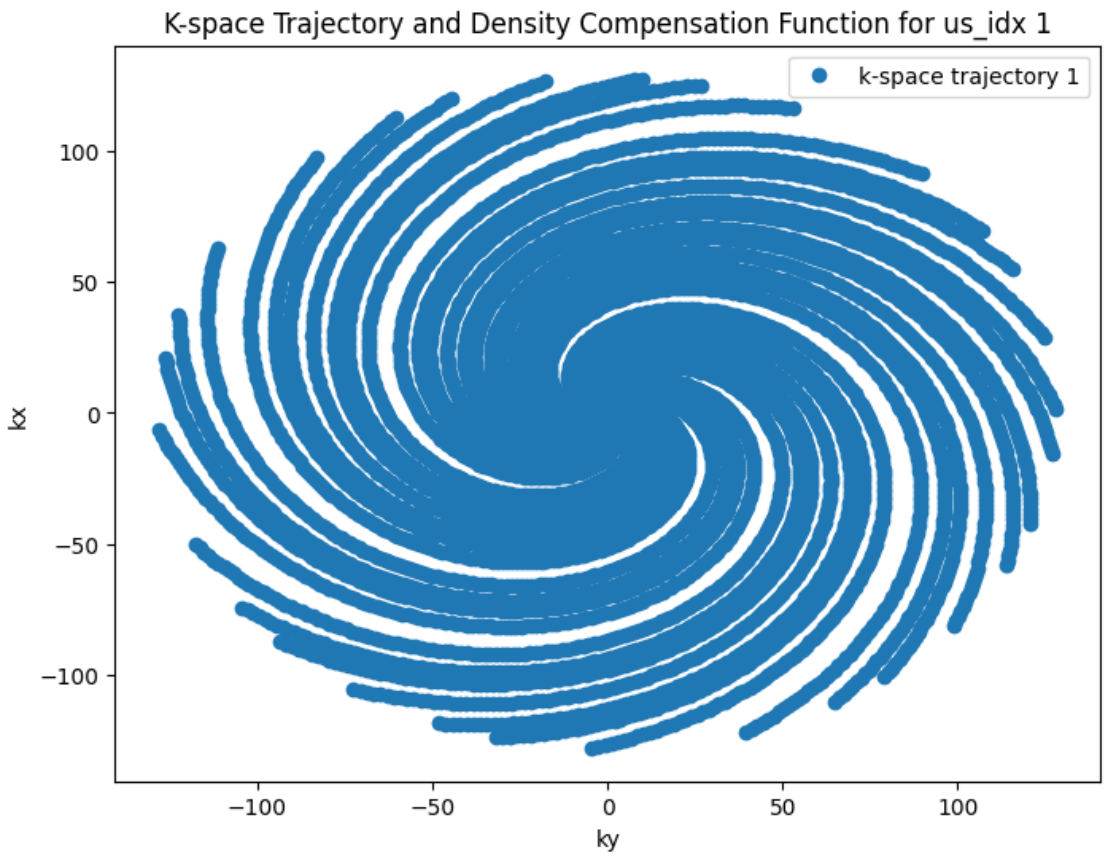


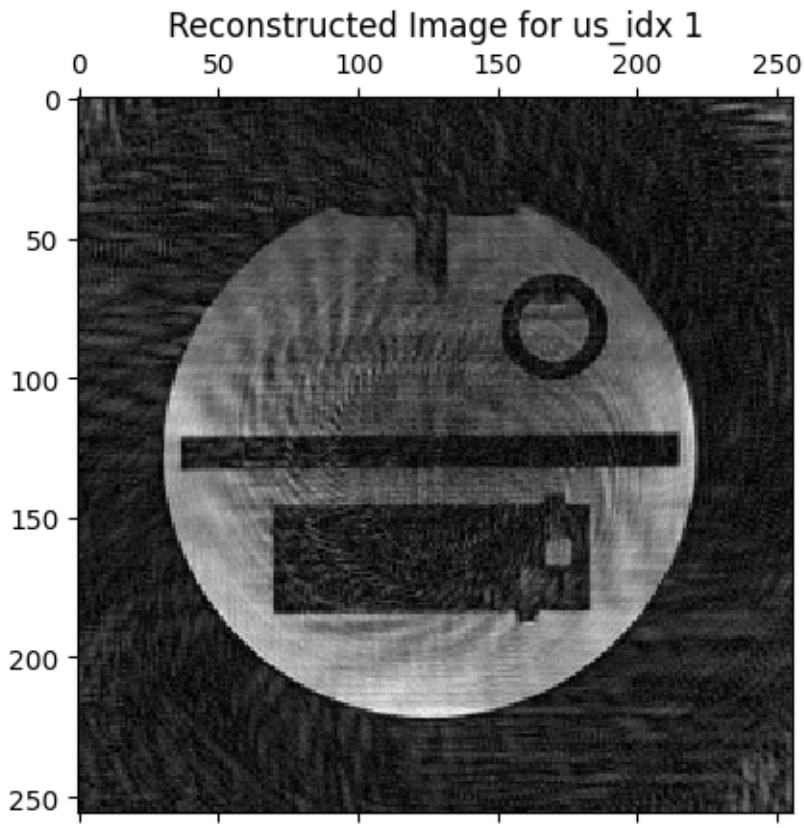
1.1.6 Comparison of K-space Trajectories and Reconstructed Images for Various Undersampling Indices

[ ]:
```
import torch
from mrpro.data import DcfData
import matplotlib.pyplot as plt

# Define values to compare for us_idx
us_idx_values = [
    torch.arange(0, 30, 1)[None, :],
    torch.arange(0, 101, 1)[None, :],
    torch.arange(0, 127, 2)[None, :]
```

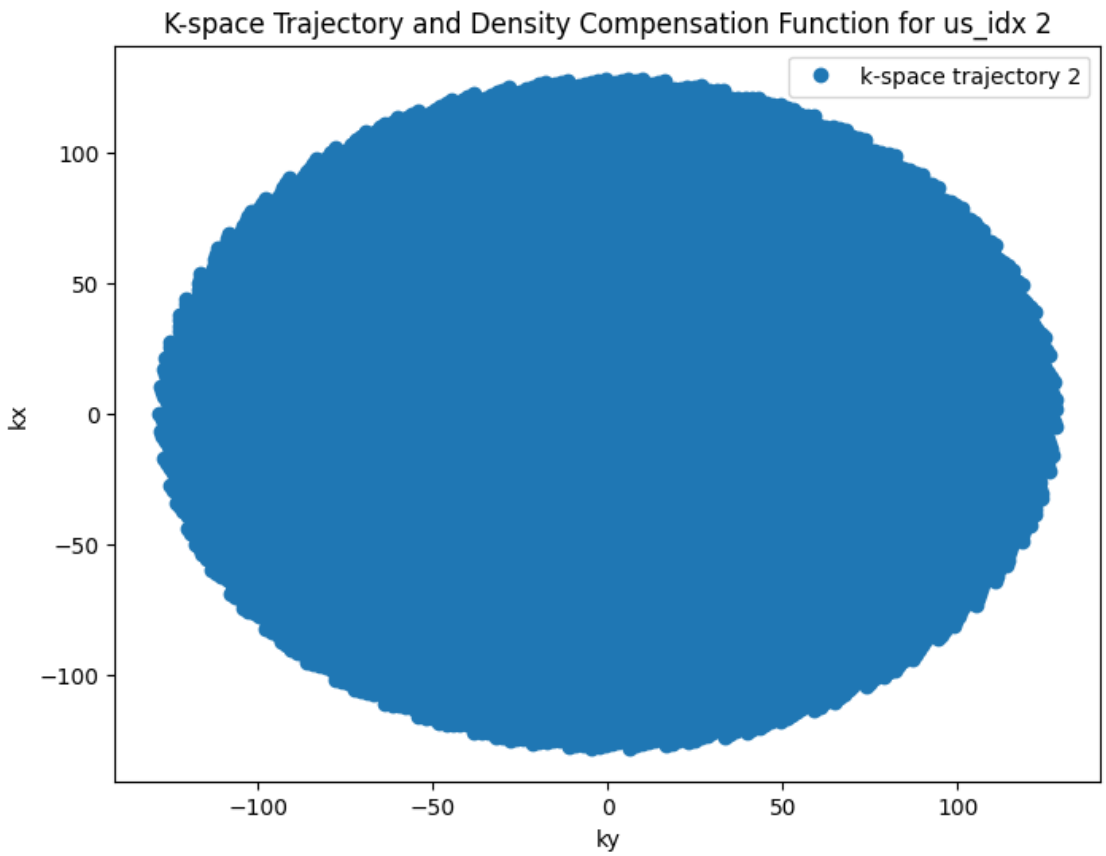
```
]

# For each us_idx value, run the reconstruction pipeline and display the␣
↪k-space trajectory
for i, us_idx in enumerate(us_idx_values):
    print(f"Calculating for us_idx {i+1}:", us_idx)

    # Split k-space data into other dimensions based on undersampling indices
    kdata_us = KData.split_k1_into_other(shifted_kdatapuls, us_idx,␣
↪other_label='repetition')

    # Calculate density compensation function (DCF)
    kdcf = DcfData.from_traj_voronoi(kdata_us.traj)

    # Perform the reconstruction
    reconstruction = mrpro.algorithms.reconstruction.DirectReconstruction.
↪from_kdata(kdata_us)
    img = reconstruction(kdata_us)

    # Plot k-space trajectory
    plt.figure(figsize=(8, 6))
    plt.plot(kdata_us.traj.ky.flatten(), kdata_us.traj.kx.flatten(), 'o',␣
↪label=f'k-space trajectory {i+1}')

    # Add labels and legend
    plt.xlabel('ky')
    plt.ylabel('kx')
    plt.title(f'K-space Trajectory and Density Compensation Function for us_idx␣
↪{i+1}')
    plt.legend()

    # Show plot
    plt.show()

    # Display the reconstructed image
    first_img = img.rss().cpu()[0, 0, :, :]  # images, z, y, x
    plt.matshow(first_img, cmap='gray')
    plt.title(f'Reconstructed Image for us_idx {i+1}')
    plt.show()
```

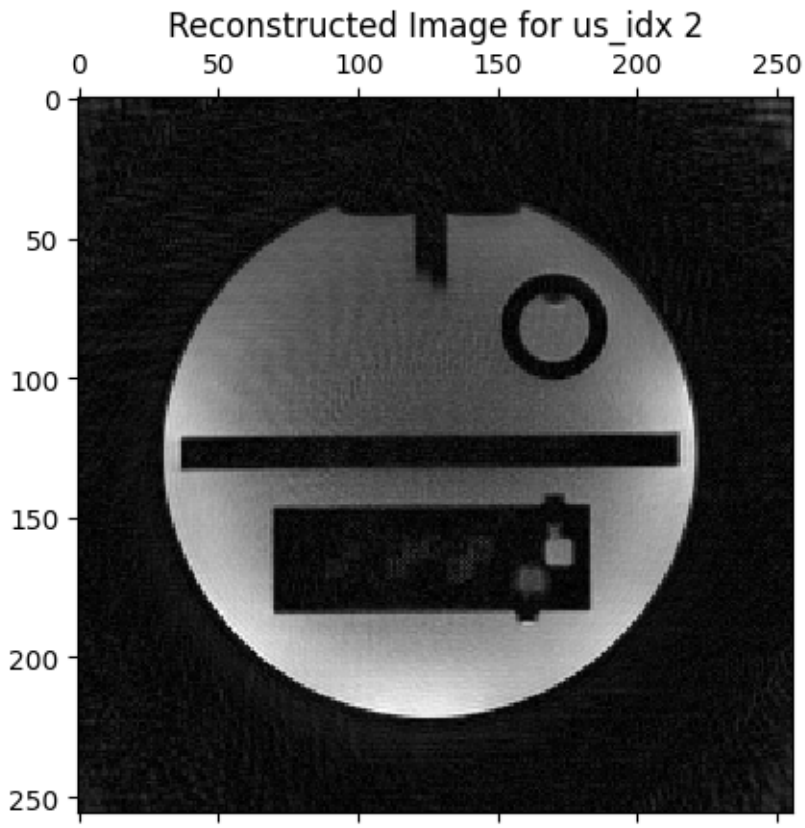
```
Calculating for us_idx 1: tensor([[ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10,
11, 12, 13, 14, 15, 16, 17,
         18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29]])
```


K-space Trajectory and Density Compensation Function for us_idx 1
k-space trajectory 1
100
50
0
−50
−100
kx
−100
−50
0
50
100
ky

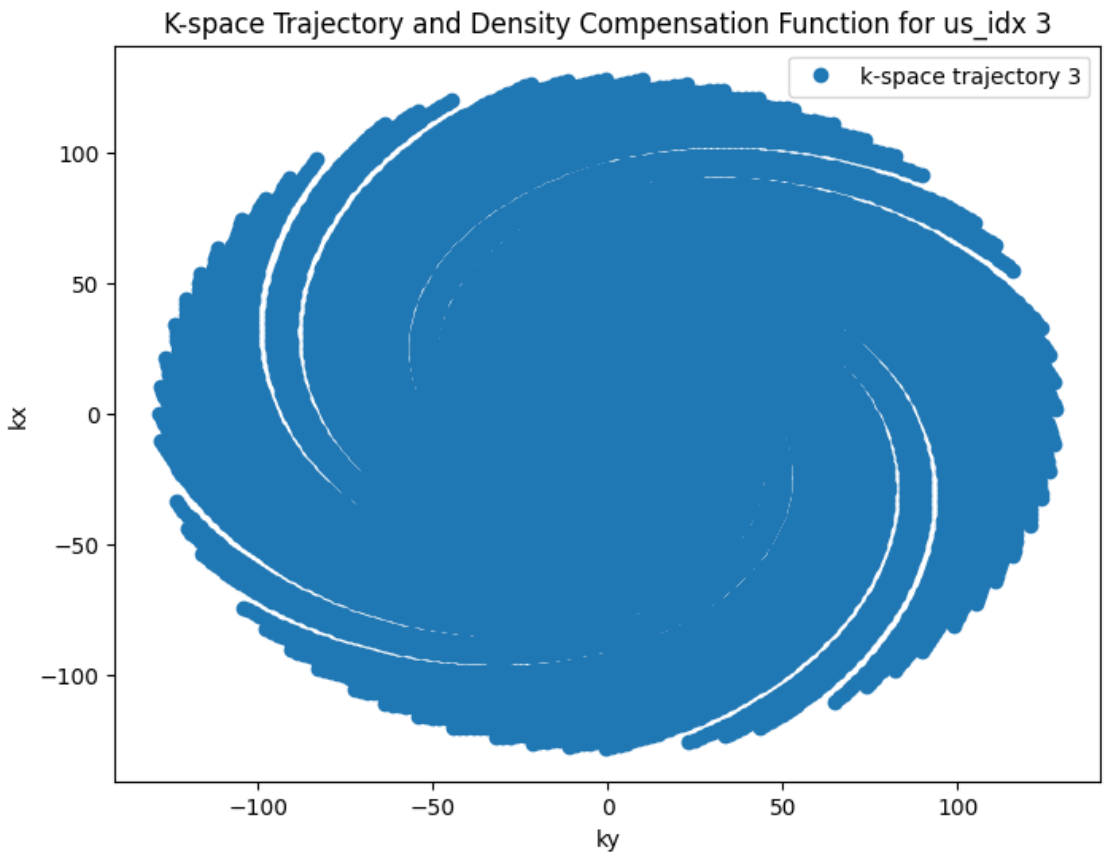


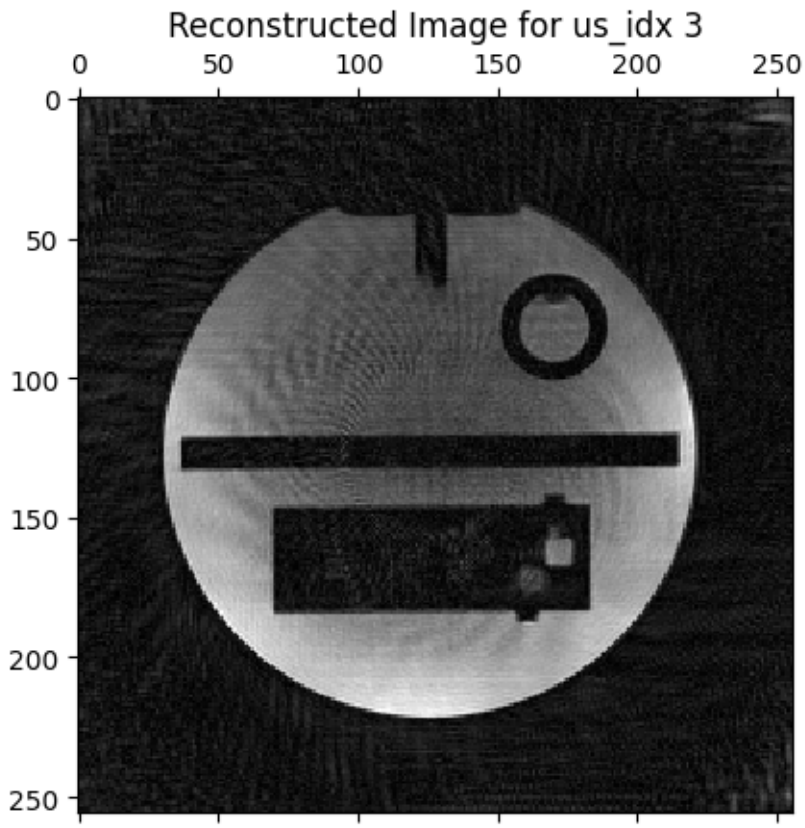

```
Calculating for us_idx 2: tensor([[  0,   1,   2,   3,   4,   5,   6,   7,   8,
9,  10,  11,  12,  13,
          14,  15,  16,  17,  18,  19,  20,  21,  22,  23,  24,  25,  26,  27,
          28,  29,  30,  31,  32,  33,  34,  35,  36,  37,  38,  39,  40,  41,
          42,  43,  44,  45,  46,  47,  48,  49,  50,  51,  52,  53,  54,  55,
          56,  57,  58,  59,  60,  61,  62,  63,  64,  65,  66,  67,  68,  69,
          70,  71,  72,  73,  74,  75,  76,  77,  78,  79,  80,  81,  82,  83,
          84,  85,  86,  87,  88,  89,  90,  91,  92,  93,  94,  95,  96,  97,
          98,  99, 100]])
```


K-space Trajectory and Density Compensation Function for us_idx 2
k-space trajectory 2
100
50
0
−50
−100
kx
−100
−50
0
50
100
ky




```
Calculating for us_idx 3: tensor([[  0,   2,   4,   6,   8,  10,  12,  14,  16,
18,  20,  22,  24,  26,
          28,  30,  32,  34,  36,  38,  40,  42,  44,  46,  48,  50,  52,  54,
          56,  58,  60,  62,  64,  66,  68,  70,  72,  74,  76,  78,  80,  82,
          84,  86,  88,  90,  92,  94,  96,  98, 100, 102, 104, 106, 108, 110,
         112, 114, 116, 118, 120, 122, 124, 126]])
```


K-space Trajectory and Density Compensation Function for us_idx 3
k-space trajectory 3
100
50
0
−50
−100
kx
−100
−50
0
50
100
ky



1.2 Nothing

1.2.1 Behind the scenes

These steps are done in a direct reconstruction:

[ ]:
```
# Calculate dcf using the trajectory
dcf_operator = mrpro.data.DcfData.from_traj_voronoi(kdata.traj).as_operator()

# Define Fourier operator using the trajectory
# and header information in kdata
fourier_operator = dcf_operator @ mrpro.operators.FourierOp.from_kdata(kdata)
adjoint_operator = fourier_operator.H

# Calculate coil maps
# Note that operators return a tuple of tensors, so we need to unpack it,
# even though there is only one tensor returned from adjoint operator.
img_coilwise = mrpro.data.IData.from_tensor_and_kheader(*adjoint_operator(kdata.
 ↪data), kdata.header)
csm_operator = mrpro.data.CsmData.from_idata_walsh(img_coilwise).as_operator()
```

```
# Perform Direct Reconstruction
forward_operator = fourier_operator @ csm_operator
adjoint_operator = forward_operator.H
img_manual = mrpro.data.IData.from_tensor_and_kheader(*adjoint_operator(kdata.
 ↪data), kdata.header)
```

1.2.2 Further behind the scenes

... these steps are equivalent to:

[ ]:
```
# Define Fourier operator
fourier_operator = mrpro.operators.FourierOp(
    recon_matrix=kdata.header.recon_matrix,
    encoding_matrix=kdata.header.encoding_matrix,
    traj=kdata.traj,
)

# Calculate dcf using the trajectory
dcf_data = mrpro.data.DcfData.from_traj_voronoi(kdata.traj)

# Perform adjoint Fourier transform
# Note that operators return a tuple of tensors, so we need to unpack it.
(img_tensor_coilwise,) = fourier_operator.adjoint(kdata.data * dcf_data.data.
 ↪unsqueeze(-4))
img_coilwise = mrpro.data.IData.from_tensor_and_kheader(img_tensor_coilwise,␣
 ↪kdata.header)


# Calculate and apply coil maps
csm_data = mrpro.data.CsmData.from_idata_walsh(img_coilwise)
csm_operator = mrpro.operators.SensitivityOp(csm_data)
(img_tensor_coilcombined,) = csm_operator.adjoint(img_tensor_coilwise)

img_more_manual = mrpro.data.IData.
 ↪from_tensor_and_kheader(img_tensor_coilcombined, kdata.header)
```

1.2.3 Check for equal results

The 3 versions result should in the same image data.

[ ]:
```
# import torch

## If the assert statement did not raise an exception, the results are equal.
# assert torch.allclose(img.data, img_manual.data)
# assert torch.allclose(img.data, img_more_manual.data)
```

[ ]:
```
# Clean-up by removing temporary directory
# shutil.rmtree(data_folder)
```

Copyright 2024 Physikalisch-Technische Bundesanstalt Apache License 2.0. See LICENSE file for details.